

```

1: // === binpow.cpp ===
2: const int mod = 998244353;
3:
4: int binpow(int a, int b, long long m = mod) {
5:     a %= m;
6:     int res = 1;
7:     while (b > 0) {
8:         if (b & 1) res = res * a % m;
9:         a = a * a % m;
10:        b >>= 1;
11:    }
12:    return res%m;
13: }
14: // binpow(number,mod-2,mod) gives mod inverse if mod is prime
15: // === combinatorics.cpp ===
16: template<typename T, const int P>
17: class combinatorics {
18:     //combinatorics<mint,2> c(200);
19:     //2 is the number whose power will be precomputed
20: public:
21:     int n;
22:     vector<T> inv, fac, ifac, pw;
23:     combinatorics(int n) : n(n), inv(n+1), fac(n+1), ifac(n+1), pw(n+1) {
24:         fac[0] = inv[0] = ifac[0] = pw[0] = T(1);
25:
26:         for(int i = 1; i <= n; i++)
27:             inv[i] = T(1)/T(i), fac[i] = fac[i - 1] * T(i), ifac[i] = ifac[i - 1]
* inv[i], pw[i] = pw[i - 1] * T(P);
28:     }
29:
30:     T ncr(int n, int r) {
31:         if(n < r or r < 0)
32:             return 0;
33:         return fac[n] * ifac[r] * ifac[n - r];
34:     }
35: };
36: // === dsu.cpp ===
37: const int N = 3e5;
38: int parent[N], sz[N];
39:
40: void make_set(int v) {
41:     parent[v] = v;
42:     sz[v] = 1;
43: }
44:
45: int find_set(int v) {
46:     if (v == parent[v])
47:         return v;
48:     return parent[v] = find_set(parent[v]);
49: }
50:
51: void union_sets(int a, int b) {
52:     a = find_set(a);
53:     b = find_set(b);
54:     if (a != b) {
55:         if (sz[a] < sz[b])
56:             swap(a, b);
57:         parent[b] = a;
58:         sz[a] += sz[b];
59:     }
60: }
61: // === egcd.cpp ===
62: // ax + by = gcd
63: int egcd(int a, int b, int& x, int& y) {
64:     x = 1, y = 0;
65:     int x1 = 0, y1 = 1, a1 = a, b1 = b;
66:     while (b1) {
67:         int q = a1 / b1;
68:         tie(x, x1) = make_tuple(x1, x - q * x1);

```

```

69:         tie(y, y1) = make_tuple(y1, y - q * y1);
70:         tie(a1, b1) = make_tuple(b1, a1 - q * b1);
71:     }
72:     return a1; //gcd
73: }
74:
75: // if gcd(a,m) = 1;
76: // ax + my = 1 => x is the mod inverse of a with mod m
77:
78: int modinv(int a, int m) {
79:     int x, y;
80:     int g = egcd(a, m, x, y);
81:     if(g==1) return (x%m + m)%m;
82:     else return -1;
83: }
84: // === fenwick_tree.cpp ===
85: struct Fenwick {
86:     vector<int> bit;
87:     int n;
88:
89:     Fenwick(int n) {
90:         this->n = n + 1;
91:         bit.assign(n + 1, 0);
92:     }
93:
94:     Fenwick(vector<int> a)
95:         : Fenwick(a.size()) {
96:         for (size_t i = 0; i < a.size(); i++)
97:             add(i, a[i]);
98:     }
99:
100:    int sum(int idx) {
101:        int ret = 0;
102:        for (++idx; idx > 0; idx -= idx & -idx)
103:            ret += bit[idx];
104:        return ret;
105:    }
106:
107:    int sum(int l, int r) {
108:        return sum(r) - sum(l - 1);
109:    }
110:
111:    void add(int idx, int delta) {
112:        for (++idx; idx < n; idx += idx & -idx)
113:            bit[idx] += delta;
114:    }
115: };
116: // === lazysegtree.cpp ===
117: // MODIFY THIS SECTION, UPDATE SECTION and PUSH FUNCTION
118: struct Node {
119:     long long sum;
120:     long long max_val;
121:     long long sz;
122:
123:     // Identity Element
124:     static Node neutral() {
125:         return {0, (long long)-4e18, 0};
126:     }
127:
128:     // Leaf Initialization: How to create a node from a single array value
129:     static Node from_value(long long v) {
130:         return {v, v, 1};
131:     }
132: };
133:
134: // MERGE
135: Node merge(const Node& a, const Node& b) {
136:     // If one side is empty/neutral (size 0), return the other
137:     if (a.sz == 0) return b;

```

```

138:     if (b.sz == 0) return a;
139:
140:     return {
141:         a.sum + b.sum,
142:         max(a.max_val, b.max_val),
143:         a.sz + b.sz
144:     };
145: }
146:
147: // TAG
148: struct Tag {
149:     long long add;
150:
151:     // Identity Tag
152:     static Tag neutral() {
153:         return {0};
154:     }
155: };
156:
157: // APPLY TAG
158: void apply_tag(Node& node, const Tag& tag) {
159:     if (node.sz == 0) return;
160:     node.sum += tag.add * node.sz; // Range Add logic for Sum
161:     node.max_val += tag.add;      // Range Add logic for Max
162: }
163:
164: // COMBINE TAGS: How to merge a new tag onto an existing one
165: void combine_tags(Tag& old_tag, const Tag& new_tag) {
166:     old_tag.add += new_tag.add;
167: }
168:
169: // CHECK (Descent): Predicate for finding an index
170: // returns first index where check is true
171: bool check(const Node& node, long long k) {
172:     return node.max_val >= k;
173: }
174:
175: struct LazySegTree {
176:     int n;
177:     vector<Node> tree;
178:     vector<Tag> lazy;
179:
180:     LazySegTree(const vector<int>& a) {
181:         n = a.size();
182:         tree.resize(4 * n, Node::neutral());
183:         lazy.resize(4 * n, Tag::neutral());
184:
185:         build(a, 1, 0, n-1);
186:     }
187:
188:     void push(int node) {
189:         //MODIFY THIS
190:         if (lazy[node].add == 0) return;
191:
192:         apply_tag(tree[2 * node], lazy[node]);
193:         combine_tags(lazy[2 * node], lazy[node]);
194:
195:         apply_tag(tree[2 * node + 1], lazy[node]);
196:         combine_tags(lazy[2 * node + 1], lazy[node]);
197:
198:         lazy[node] = Tag::neutral();
199:     }
200:
201:
202:     void build(const vector<int>& a, int node, int start, int end) {
203:         if (start == end) {
204:             tree[node] = Node::from_value(a[start]);
205:         } else {
206:             int mid = (start + end) / 2;

```

```

207:             build(a, 2 * node, start, mid);
208:             build(a, 2 * node + 1, mid + 1, end);
209:             tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
210:         }
211:     }
212:
213:     void update(int l, int r, long long val, int node, int start, int end) {
214:         if (start > end || start > r || end < l) return;
215:
216:         if (start >= l && end <= r) {
217:             //MODIFY THIS
218:             Tag new_tag = {val};
219:
220:             apply_tag(tree[node], new_tag);
221:             combine_tags(lazy[node], new_tag);
222:             return;
223:         }
224:
225:
226:         push(node);
227:         int mid = (start + end) / 2;
228:         update(l, r, val, 2 * node, start, mid);
229:         update(l, r, val, 2 * node + 1, mid + 1, end);
230:         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
231:     }
232:
233:     Node query(int l, int r, int node, int start, int end) {
234:         if (start > end || start > r || end < l) return Node::neutral();
235:         if (start >= l && end <= r) return tree[node];
236:
237:         push(node);
238:         int mid = (start + end) / 2;
239:         return merge(query(l, r, 2 * node, start, mid),
240:                     query(l, r, 2 * node + 1, mid + 1, end));
241:     }
242:
243:     // Descent: Find first index in [l, r] satisfying check()
244:     int find_first(int l, int r, long long k, int node, int start, int end) {
245:
246:         if (start > end || start > r || end < l || !check(tree[node], k)) {
247:             return -1;
248:         }
249:
250:         if (start == end) return start;
251:
252:         push(node);
253:         int mid = (start + end) / 2;
254:
255:         int res = find_first(l, r, k, 2 * node, start, mid);
256:
257:         if (res == -1) {
258:             res = find_first(l, r, k, 2 * node + 1, mid + 1, end);
259:         }
260:
261:         return res;
262:     }
263:
264:     void build(const vector<int>& a) { build(a, 1, 0, n - 1); }
265:     void update(int l, int r, long long val) { update(l, r, val, 1, 0, n - 1); }
266:     Node query(int l, int r) { return query(l, r, 1, 0, n - 1); }
267:     int find_first(int l, int r, long long k) { return find_first(l, r, k, 1, 0, n
- 1); }
268: };
269: //LazySegTree st(arr);
270: // === linear_sieve.cpp ===
271: const int N = 10000000;
272: vector<int> lp(N+1); //lowest prime factor
273: vector<int> pr; // prime list
274:

```

```

275: void sieve(){
276:     for(int i = 2; i <= N; ++i){
277:         if (lp[i] == 0) {
278:             lp[i] = i;
279:             pr.push_back(i);
280:         }
281:         for (int j = 0; i * pr[j] <= N; ++j) {
282:             lp[i * pr[j]] = pr[j];
283:             if (pr[j] == lp[i]) {
284:                 break;
285:             }
286:         }
287:     }
288: }
289:
290: // === modint.cpp ===
291: #include<bits/stdc++.h>
292: using namespace std;
293:
294: #define int int
295: template<const int &MOD>
296: struct _m_int {
297:     int val;
298:
299:     _m_int(int64_t v = 0) {
300:         if (v < 0) v = v % MOD + MOD;
301:         if (v >= MOD) v %= MOD;
302:         val = int(v);
303:     }
304:
305:     _m_int(uint64_t v) {
306:         if (v >= MOD) v %= MOD;
307:         val = int(v);
308:     }
309:
310:     _m_int(int v) : _m_int(int64_t(v)) {}
311:     _m_int(unsigned v) : _m_int(uint64_t(v)) {}
312:
313:     explicit operator int() const { return val; }
314:     explicit operator unsigned() const { return val; }
315:     explicit operator int64_t() const { return val; }
316:     explicit operator uint64_t() const { return val; }
317:     explicit operator double() const { return val; }
318:     explicit operator long double() const { return val; }
319:
320:     _m_int& operator+=(const _m_int &other) {
321:         val -= MOD - other.val;
322:         if (val < 0) val += MOD;
323:         return *this;
324:     }
325:
326:     _m_int& operator-=(const _m_int &other) {
327:         val -= other.val;
328:         if (val < 0) val += MOD;
329:         return *this;
330:     }
331:
332:     static unsigned fast_mod(uint64_t x, unsigned m = MOD) {
333:     #if !defined(_WIN32) || defined(_WIN64)
334:         return unsigned(x % m);
335:     #endif
336:         // Optimized mod for Codeforces 32-bit machines.
337:         // x must be less than 2^32 * m for this to work, so that x / m fits in an
338:         unsigned x_high = unsigned(x >> 32), x_low = unsigned(x);
339:         unsigned quot, rem;
340:         asm("divl %4\n"
341:             : "=a" (quot), "=d" (rem)
342:             : "d" (x_high), "a" (x_low), "r" (m));

```

```

343:         return rem;
344:     }
345:
346:     _m_int& operator*=(const _m_int &other) {
347:         val = fast_mod(uint64_t(val) * other.val);
348:         return *this;
349:     }
350:
351:     _m_int& operator/=(const _m_int &other) {
352:         return *this * other.inv();
353:     }
354:
355:     friend _m_int operator+(const _m_int &a, const _m_int &b) { return _m_int(a) +
356:     = b; }
357:     friend _m_int operator-(const _m_int &a, const _m_int &b) { return _m_int(a) -
358:     = b; }
359:     friend _m_int operator*(const _m_int &a, const _m_int &b) { return _m_int(a) *
360:     = b; }
361:     friend _m_int operator/(const _m_int &a, const _m_int &b) { return _m_int(a) /
362:     = b; }
363:
364:     _m_int& operator++() {
365:         val = val == MOD - 1 ? 0 : val + 1;
366:         return *this;
367:     }
368:
369:     _m_int& operator--() {
370:         val = val == 0 ? MOD - 1 : val - 1;
371:         return *this;
372:     }
373:
374:     _m_int operator++(int) { _m_int before = *this; ++*this; return before; }
375:     _m_int operator--(int) { _m_int before = *this; --*this; return before; }
376:
377:     _m_int operator-() const {
378:         return val == 0 ? 0 : MOD - val;
379:     }
380:
381:     friend bool operator==(const _m_int &a, const _m_int &b) { return a.val == b.val; }
382:     friend bool operator!=(const _m_int &a, const _m_int &b) { return a.val != b.val; }
383:     friend bool operator<(const _m_int &a, const _m_int &b) { return a.val < b.val; }
384:     friend bool operator>(const _m_int &a, const _m_int &b) { return a.val > b.val; }
385:     friend bool operator<=(const _m_int &a, const _m_int &b) { return a.val <= b.val; }
386:     friend bool operator>=(const _m_int &a, const _m_int &b) { return a.val >= b.val; }
387:
388:     static const int SAVE_INV = int(1e6) + 5;
389:     static _m_int save_inv[SAVE_INV];
390:
391:     static void prepare_inv() {
392:         // Ensures that MOD is prime, which is necessary for the inverse algorithm
393:         below.
394:         for (int64_t p = 2; p * p <= MOD; p += p % 2 + 1)
395:             assert(MOD % p != 0);
396:
397:         save_inv[0] = 0;
398:         save_inv[1] = 1;
399:
400:         for (int i = 2; i < SAVE_INV; i++)
401:             save_inv[i] = save_inv[MOD % i] * (MOD - MOD / i);
402:     }
403:
404:     _m_int inv() const {
405:         if (save_inv[1] == 0)

```

```

401:         prepare_inv();
402:
403:         if (val < SAVE_INV)
404:             return save_inv[val];
405:
406:         _m_int product = 1;
407:         int v = val;
408:
409:         do {
410:             product *= MOD - MOD / v;
411:             v = MOD % v;
412:         } while (v >= SAVE_INV);
413:
414:         return product * save_inv[v];
415:     }
416:
417:     _m_int pow(int64_t p) const {
418:         if (p < 0)
419:             return inv().pow(-p);
420:
421:         _m_int a = *this, result = 1;
422:
423:         while (p > 0) {
424:             if (p & 1)
425:                 result *= a;
426:
427:             p >>= 1;
428:
429:             if (p > 0)
430:                 a *= a;
431:         }
432:
433:         return result;
434:     }
435:
436:     friend ostream& operator<<(ostream &os, const _m_int &m) {
437:         return os << m.val;
438:     }
439: };
440:
441: template<const int &MOD> _m_int<MOD> _m_int<MOD>::save_inv[_m_int<MOD>::SAVE_INV];
442:
443: const int MOD = ;
444: using mint = _m_int<MOD>;
445: void __print(mint x) { cerr << x; }
446: // === multiple_string_hashes.cpp ===
447: /////////////// !!!!!!!!!!!!! 1-based indexing !!!!!!!!!!!!! ///////////////
448:
449: mt19937_64 RNG(chrono::steady_clock::now().time_since_epoch().count());
450:
451: struct Hash{
452:     int b, n; // b = number of hashes ; n = length of string
453:     const int mod = 1e9 + 7;
454:     vector<vector<int>> fw, bc, pb, ib;
455:     vector<int> bases;
456:
457:     inline int power(int x, int y){
458:         if (y == 0){
459:             return 1;
460:         }
461:
462:         int v = power(x, y / 2);
463:         v = 1LL * v * v % mod;
464:         if (y & 1) return 1LL * v * x % mod;
465:         else return v;
466:     }
467:
468:     inline void init(int nn, int bb, string str){
469:         n = nn;

```

```

470:         b = bb;
471:         fw = vector<vector<int>>(b, vector<int>(n + 2, 0));
472:         bc = vector<vector<int>>(b, vector<int>(n + 2, 0));
473:         pb = vector<vector<int>>(b, vector<int>(n + 2, 1));
474:         ib = vector<vector<int>>(b, vector<int>(n + 2, 1));
475:         bases = vector<int>(b);
476:         str = "0" + str;
477:
478:         for (auto &x : bases) x = RNG() % (mod / 10);
479:
480:         for (int i = 0; i < b; i++){
481:             for (int j = 1; j <= n + 1; j++){
482:                 pb[i][j] = 1LL * pb[i][j - 1] * bases[i] % mod;
483:             }
484:             ib[i][n + 1] = power(pb[i][n + 1], mod - 2);
485:             for (int j = n; j >= 1; j--){
486:                 ib[i][j] = 1LL * ib[i][j + 1] * bases[i] % mod;
487:             }
488:
489:             for (int j = 1; j <= n; j++){
490:                 fw[i][j] = (fw[i][j - 1] + 1LL * (str[j] - 'a' + 1) * pb[i][j]) %
mod;
491:             }
492:             for (int j = n; j >= 1; j--){
493:                 bc[i][j] = (bc[i][j + 1] + 1LL * (str[j] - 'a' + 1) * pb[i][n + 1
- j]) % mod;
494:             }
495:         }
496:     }
497:
498:     inline int getfwhash(int l, int r, int i){
499:         int ans = fw[i][r] - fw[i][l - 1];
500:         ans = 1LL * ans * ib[i][l - 1] % mod;
501:
502:         if (ans < 0) ans += mod;
503:
504:         return ans;
505:     }
506:
507:     inline int getbchash(int l, int r, int i){
508:         int ans = bc[i][l] - bc[i][r + 1];
509:         ans = 1LL * ans * ib[i][n - r] % mod;
510:
511:         if (ans < 0) ans += mod;
512:
513:         return ans;
514:     }
515:
516:     inline bool equal(int l1, int r1, int l2, int r2){
517:         for (int i = 0; i < b; i++){
518:             int v1, v2;
519:             if (l1 <= r1) v1 = getfwhash(l1, r1, i);
520:             else v1 = getbchash(r1, l1, i);
521:
522:             if (l2 <= r2) v2 = getfwhash(l2, r2, i);
523:             else v2 = getbchash(r2, l2, i);
524:
525:             if (v1 != v2) return false;
526:         }
527:         return true;
528:     }
529:
530:     inline bool pal(int l, int r){
531:         return equal(l, r, r, l);
532:     }
533: };
534:
535: // Hash h;
536: // h.init(n,2,str); (length,no of hashes,string)

```

```

537: // === ordered_set.cpp ===
538: #include <ext/pb_ds/assoc_container.hpp>
539: #include <ext/pb_ds/tree_policy.hpp>
540: using namespace __gnu_pbds;
541: template<class T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tre
e_order_statistics_node_update>;
542: template<class T> using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tr
ee_tag, tree_order_statistics_node_update>;
543:
544: /** X.find_by_order(k) return kth element. 0 indexed.
545: /** X.order_of_key(k) returns count of elements strictly less than k.
546:
547:
548:
549: // === segmented_sieve.cpp ===
550: void solve() {
551:     int l, r;
552:     cin >> l >> r;
553:
554:     int lim = sqrt(r) + 1;
555:
556:     vector<int> base_primes;
557:     vector<bool> is_prime(lim + 1, 1);
558:     for (int i = 2; i * i <= r; i++) {
559:         if (!is_prime[i]) continue;
560:         for (int j = i * i; j <= lim; j += i) {
561:             // dbg(j);
562:             is_prime[j] = false;
563:         }
564:     }
565:
566:     for (int i = 2; i * i <= r; i++) if (is_prime[i]) base_primes.push_back(i);
567:
568:     vector<int> is_prime_seg(r - l + 1, 1);
569:     for (int p : base_primes) {
570:         int st = ((l + p - 1) / p) * p;
571:         for (int j = st; j <= r; j += p) is_prime_seg[j - l] = 0;
572:     }
573:     if (l == 1) is_prime_seg[0] = false;
574:     // is_prime_seg[i] = true ==> l + i is prime
575: }
576: // === segment_tree.cpp ===
577: // MODIFY THIS SECTION, BUILD and UPDATE
578: struct Node {
579:     long long sum;
580:     long long max_val;
581:
582:     static Node neutral() {
583:         return {0, (long long)-1e18};
584:     }
585: };
586:
587: // Merge logic
588: Node merge(const Node& a, const Node& b) {
589:     return {
590:         a.sum + b.sum,
591:         max(a.max_val, b.max_val)
592:     };
593: }
594:
595: // Descent Predicate
596: // we get first index from the left where the check function is true
597: bool check(const Node& node, long long k) {
598:     return node.max_val >= k;
599: }
600:
601: struct SegTree {
602:     int n;
603:     vector<Node> tree;

```

```

604:
605: SegTree(const vector<int>& a) {
606:     n = a.size();
607:     tree.resize(4 * n, Node::neutral());
608:
609:     build(a, 1, 0, n - 1);
610: }
611:
612: void build(const vector<int>& a, int node, int start, int end) {
613:     if (start == end) {
614:
615:         // MODIFY THIS
616:         // MODIFY THIS
617:         // MODIFY THIS
618:         tree[node] = { (long long)a[start], (long long)a[start] };
619:
620:     } else {
621:         int mid = (start + end) / 2;
622:         build(a, 2 * node, start, mid);
623:         build(a, 2 * node + 1, mid + 1, end);
624:         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
625:     }
626: }
627:
628: void update(int idx, int val, int node, int start, int end) {
629:     if (start == end) {
630:
631:         // MODIFY THIS
632:         // MODIFY THIS
633:         // MODIFY THIS
634:         tree[node] = { (long long)val, (long long)val };
635:
636:     } else {
637:         int mid = (start + end) / 2;
638:         if (start <= idx && idx <= mid) {
639:             update(idx, val, 2 * node, start, mid);
640:         } else {
641:             update(idx, val, 2 * node + 1, mid + 1, end);
642:         }
643:         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
644:     }
645: }
646:
647: Node query(int l, int r, int node, int start, int end) {
648:     if (r < start || end < l) {
649:         return Node::neutral();
650:     }
651:     if (l <= start && end <= r) {
652:         return tree[node];
653:     }
654:     int mid = (start + end) / 2;
655:     Node p1 = query(l, r, 2 * node, start, mid);
656:     Node p2 = query(l, r, 2 * node + 1, mid + 1, end);
657:     return merge(p1, p2);
658: }
659:
660: // Descent: Find first index in range [l, r] where check returns true
661: int find_first(int l, int r, long long k, int node, int start, int end) {
662:     if (start > r || end < l || !check(tree[node], k)) {
663:         return -1;
664:     }
665:
666:     if (start == end) {
667:         return start;
668:     }
669:
670:     int mid = (start + end) / 2;
671:     int res = -1;
672:

```

```

673:         // 2. Try Left
674:         if (mid >= 1) {
675:             res = find_first(1, r, k, 2 * node, start, mid);
676:         }
677:
678:         // 3. Try Right if not found in Left
679:         if (res == -1) {
680:             res = find_first(1, r, k, 2 * node + 1, mid + 1, end);
681:         }
682:
683:         return res;
684:     }
685:
686:     // Wrappers
687:     void build(const vector<int>& a) { build(a, 1, 0, n - 1); }
688:     void update(int idx, int val) { update(idx, val, 1, 0, n - 1); }
689:     Node query(int l, int r) { return query(l, r, 1, 0, n - 1); }
690:     int find_first(int l, int r, long long k) { return find_first(1, r, k, 1, 0, n
- 1); }
691: };
692: // SegTree st(arr);
693: // === sieve.cpp ===
694: const int MAXN = 10000000; // adjust as needed
695: vector<int> spf(MAXN+1); // smallest prime factor
696: vector<int> primelist;
697:
698: // Precompute SPF in O(n log log n)
699: void sieve() {
700:     iota(spf.begin(), spf.end(), 0); // spf[i] = i initially
701:     for (int i = 2; i * i <= MAXN; i++) {
702:         if (spf[i] == i) { // i is prime
703:             for (int j = i * i; j <= MAXN; j += i) {
704:                 if (spf[j] == j) spf[j] = i;
705:             }
706:         }
707:     }
708:     for (int i = 2; i <= MAXN; i++) {
709:         if (spf[i] == i) {
710:             primelist.push_back(i);
711:         }
712:     }
713: }
714:
715: // prime factorization in O(log n)
716: vector<pair<int,int>> getPrimeFactors(int x) {
717:     vector<pair<int,int>> res;
718:     while (x > 1) {
719:         int p = spf[x], cnt = 0;
720:         while (x % p == 0) {
721:             x /= p;
722:             cnt++;
723:         }
724:         res.push_back({p, cnt});
725:     }
726:     return res;
727: }
728:
729: // all divisors in O(log n + #divisors(n))
730: // #divisors of n = n ^ 1/3
731: vector<int> getDivisors(int x) {
732:     auto pf = getPrimeFactors(x);
733:     vector<int> divisors = {1};
734:
735:     for (auto [p, cnt] : pf) {
736:         int sz = divisors.size();
737:         long long pw = 1;
738:         for (int i = 1; i <= cnt; i++) {
739:             pw *= p;
740:             for (int j = 0; j < sz; j++) {

```

```

741:                 divisors.push_back(divisors[j] * pw);
742:             }
743:         }
744:     }
745:     sort(divisors.begin(), divisors.end());
746:     return divisors;
747: }
748:
749:
750: // === sparse_table.cpp ===
751:
752: int log2_floor(unsigned long i) {
753:     return i ? __builtin_clzll(1) - __builtin_clzll(i) : -1;
754: }
755:
756: signed main() {
757:
758:     vector<int> array = {1,5,3,1,5};
759:
760:     int MAXN = 2*1e5, K = log2_floor(MAXN), N=5;
761:     int st[K + 1][MAXN];
762:
763:     copy(array.begin(), array.end(), st[0]);
764:
765:     for (int i = 1; i <= K; i++)
766:         for (int j = 0; j + (1 << i) <= N; j++) {
767:             // st[i][j] = f(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
768:             // f is the function for range query
769:
770:             // st[i][j] = st[i - 1][j] + st[i - 1][j + (1 << (i - 1))]; --> Precompute
for Range sum queries
771:             st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]); // range min
queries
772:         }
773:
774:         // Range sum queries --> O(log N)
775:         // long long sum = 0;
776:         // for (int i = K; i >= 0; i--) {
777:         //     if ((1 << i) <= R - L + 1) {
778:         //         sum += st[i][L];
779:         //         L += 1 << i;
780:         //     }
781:         // }
782:
783:
784:         // Range min query --> O(1)
785:         int L = 1, R=4;
786:         int i = log2_floor(R - L + 1);
787:         int minimum = min(st[i][L], st[i][R - (1 << i) + 1]);
788:
789:         cout<<minimum<<endl;
790:
791:         return 0;
792:     }
793: }
794: // === trie_bit_array.cpp ===
795: // faster than normal trie but not memeroy efficinet as there is no deletions
796:
797: const int MAX_BITS = 31; // For 32-bit integers (bit positions 0â200\22330)
798:
799: struct TrieNode {
800:     int next[2] = {-1, -1}; // next[0] and next[1]
801:     int count = 0; // number of elements passing through this node
802: };
803:
804: struct BitTrie {
805:     vector<TrieNode> trie;
806:
807:     BitTrie() {

```

```

808:         trie.emplace_back(); // root node at index 0
809:     }
810:
811:     void insert(int num) {
812:         int node = 0;
813:         for (int i = MAX_BITS; i >= 0; --i) {
814:             int bit = (num >> i) & 1;
815:             if (trie[node].next[bit] == -1) {
816:                 trie[node].next[bit] = trie.size();
817:                 trie.emplace_back();
818:             }
819:             node = trie[node].next[bit];
820:             trie[node].count++;
821:         }
822:     }
823:
824:     int maxXor(int num) {
825:         int node = 0, result = 0;
826:         for (int i = MAX_BITS; i >= 0; --i) {
827:             int bit = (num >> i) & 1;
828:             int opposite = bit ^ 1;
829:             if (trie[node].next[opposite] != -1 &&
830:                 trie[trie[node].next[opposite]].count > 0) {
831:                 result |= (1 << i);
832:                 node = trie[node].next[opposite];
833:             } else if (trie[node].next[bit] != -1) {
834:                 node = trie[node].next[bit];
835:             } else {
836:                 break;
837:             }
838:         }
839:         return result;
840:     }
841: };
842:
843: // === trie_bit.cpp ===
844: struct BitTrieNode {
845:     BitTrieNode* child[2];
846:     int count; // how many numbers pass through this node
847:     BitTrieNode() {
848:         child[0] = child[1] = nullptr;
849:         count = 0;
850:     }
851: };
852:
853: class BitTrie {
854: private:
855:     BitTrieNode* root;
856:     static const int BIT_LEN = 31;
857:
858: public:
859:     BitTrie() {
860:         root = new BitTrieNode();
861:     }
862:     void insert(int x) {
863:         BitTrieNode* node = root;
864:         node->count++;
865:         for (int b = BIT_LEN - 1; b >= 0; b--) {
866:             int bit = (x >> b) & 1;
867:             if (!node->child[bit]) {
868:                 node->child[bit] = new BitTrieNode();
869:             }
870:             node = node->child[bit];
871:             node->count++;
872:         }
873:     }
874:     void remove(int x) {
875:         if (!contains(x)) {
876:             return;

```

```

877:         }
878:         BitTrieNode* node = root;
879:         node->count--;
880:         for (int b = BIT_LEN - 1; b >= 0; b--) {
881:             int bit = (x >> b) & 1;
882:             BitTrieNode* nxt = node->child[bit];
883:             nxt->count--;
884:             node = nxt;
885:         }
886:     }
887:     bool contains(int x) const {
888:         BitTrieNode* node = root;
889:         for (int b = BIT_LEN - 1; b >= 0; b--) {
890:             if (!node) return false;
891:             int bit = (x >> b) & 1;
892:             if (!node->child[bit] || node->child[bit]->count == 0) {
893:                 return false;
894:             }
895:             node = node->child[bit];
896:         }
897:         return node && node->count > 0;
898:     }
899:     int maxXor(int x) const {
900:         BitTrieNode* node = root;
901:         int result = 0;
902:         for (int b = BIT_LEN - 1; b >= 0; b--) {
903:             int bit = (x >> b) & 1;
904:             int desired = bit ^ 1;
905:             if (node->child[desired] && node->child[desired]->count > 0) {
906:                 result |= (1 << b);
907:                 node = node->child[desired];
908:             } else if (node->child[bit] && node->child[bit]->count > 0) {
909:                 node = node->child[bit];
910:             } else {
911:                 break;
912:             }
913:         }
914:         return result;
915:     }
916: };
917: // BitTrie trie;
918: // === trie.cpp ===
919: struct TrieNode {
920:     TrieNode* children[26];
921:     bool is_end;
922:     int prefix_count; // how many words pass through this node
923:     int end_count; // how many words end exactly at this node
924:     TrieNode() {
925:         is_end = false;
926:         prefix_count = 0;
927:         end_count = 0;
928:         for (int i = 0; i < 26; i++) {
929:             children[i] = nullptr;
930:         }
931:     }
932: };
933: class Trie {
934: private:
935:     TrieNode* root;
936:
937: public:
938:     Trie() {
939:         root = new TrieNode();
940:     }
941:
942:     void insert(const string &word) {
943:         TrieNode* node = root;
944:         node->prefix_count++;
945:         for (char ch : word) {

```

```

946:         int idx = ch - 'a';
947:         if (idx < 0 || idx >= 26) {
948:             continue;
949:         }
950:         if (!node->children[idx]) {
951:             node->children[idx] = new TrieNode();
952:         }
953:         node = node->children[idx];
954:         node->prefix_count++;
955:     }
956:     node->is_end = true;
957:     node->end_count++;
958: }
959:
960: bool search(const string &word) const {
961:     TrieNode* node = root;
962:     for (char ch : word) {
963:         int idx = ch - 'a';
964:         if (idx < 0 || idx >= 26 || !node->children[idx]) {
965:             return false;
966:         }
967:         node = node->children[idx];
968:     }
969:     return node->is_end;
970: }
971: bool startsWith(const string &prefix) const {
972:     TrieNode* node = root;
973:     for (char ch : prefix) {
974:         int idx = ch - 'a';
975:         if (idx < 0 || idx >= 26 || !node->children[idx]) {
976:             return false;
977:         }
978:         node = node->children[idx];
979:     }
980:     return true;
981: }
982: int countWordsEqualTo(const string &word) const {
983:     TrieNode* node = root;
984:     for (char ch : word) {
985:         int idx = ch - 'a';
986:         if (idx < 0 || idx >= 26 || !node->children[idx]) {
987:             return 0;
988:         }
989:         node = node->children[idx];
990:     }
991:     return node->end_count;
992: }
993: int countWordsStartingWith(const string &prefix) const {
994:     TrieNode* node = root;
995:     for (char ch : prefix) {
996:         int idx = ch - 'a';
997:         if (idx < 0 || idx >= 26 || !node->children[idx]) {
998:             return 0;
999:         }
1000:         node = node->children[idx];
1001:     }
1002:     return node->prefix_count;
1003: }
1004: void remove(const string &word) {
1005:     TrieNode* node = root;
1006:     vector<TrieNode*> path;
1007:     path.reserve(word.size() + 1);
1008:     path.push_back(root);
1009:
1010:     for (char ch : word) {
1011:         int idx = ch - 'a';
1012:         if (idx < 0 || idx >= 26 || node->children[idx] == nullptr) {
1013:             return;
1014:         }

```

```

1015:         node = node->children[idx];
1016:         path.push_back(node);
1017:     }
1018:     if (node->end_count == 0) {
1019:         return;
1020:     }
1021:
1022:     node->end_count--;
1023:     if (node->end_count == 0) {
1024:         node->is_end = false;
1025:     }
1026:
1027:     for (TrieNode* nd : path) {
1028:         nd->prefix_count--;
1029:     }
1030:
1031: }
1032: };
1033: // Trie trie;
1034: // === unordered_map_custom_hash.cpp ===
1035: struct custom_hash {
1036:     static uint64_t splitmix64(uint64_t x) {
1037:         x += 0x9e3779b97f4a7c15;
1038:         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
1039:         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
1040:         return x ^ (x >> 31);
1041:     }
1042:     size_t operator()(uint64_t x) const {
1043:         static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
1044:         return splitmix64(x + FIXED_RANDOM);
1045:     }
1046: };
1047: // === xorbasis.cpp ===
1048: template<typename T = long long>
1049: struct XorBasis {
1050:     static const int B = 62;
1051:     T basis[B];
1052:     int sz = 0;
1053:
1054:     XorBasis() {
1055:         memset(basis, 0, sizeof(basis));
1056:         sz = 0;
1057:     }
1058:
1059:     void clear() {
1060:         memset(basis, 0, sizeof(basis));
1061:         sz = 0;
1062:     }
1063:     void insert(T x) {
1064:         for (int i = B - 1; i >= 0; i--) {
1065:             if ((x >> i) & 1) {
1066:                 if (!basis[i]) {
1067:                     basis[i] = x;
1068:                     sz++;
1069:                     return;
1070:                 }
1071:                 x ^= basis[i];
1072:             }
1073:         }
1074:     }
1075:
1076:     // Is there any subset having xor = x
1077:     bool contains(T x) {
1078:         for (int i = B - 1; i >= 0; i--) {
1079:             if ((x >> i) & 1) {
1080:                 if (!basis[i]) return false;
1081:                 x ^= basis[i];
1082:             }

```



```

1083:     }
1084:     return x == 0;
1085: }
1086:
1087: // Max subset xor after xoring starting it with x
1088: T max_xor(T x = 0) {
1089:     for (int i = B - 1; i >= 0; i--) {
1090:         x = max(x, x ^ basis[i]);
1091:     }
1092:     return x;
1093: }
1094:
1095: // Min subset xor after xoring starting it with x
1096: T min_xor(T x = 0) {
1097:     for (int i = B - 1; i >= 0; i--) {
1098:         x = min(x, x ^ basis[i]);
1099:     }
1100:     return x;
1101: }
1102:
1103: // Kth smallest subset xor (1-based index)
1104: // 1st smallest is 0. Returns -1 if k > 2^sz.
1105: T kth_smallest(T k) {
1106:     T total = (T)1 << sz;
1107:     if (k < 1 || k > total) return -1;
1108:
1109:     T res = 0;
1110:     T cnt = total;
1111:
1112:     // Iterate MSB to LSB
1113:     for (int i = B - 1; i >= 0; i--) {
1114:         if (basis[i]) {
1115:             cnt >>= 1;
1116:             if (k > cnt) {
1117:                 if (!(res >> i) & 1) res ^= basis[i];
1118:                 k -= cnt;
1119:             }
1120:             else {
1121:                 if ((res >> i) & 1) res ^= basis[i];
1122:             }
1123:         }
1124:     }
1125:     return res;
1126: }
1127:
1128: // Kth largest subset xor
1129: T kth_largest(T k) {
1130:     T total = (T)1 << sz;
1131:     if (k < 1 || k > total) return -1;
1132:     return kth_smallest(total - k + 1);
1133: }
1134:
1135: // Number of subsets having xor < x
1136: T count_less(T x) {
1137:     if (x < 0) return 0;
1138:
1139:     T res = 0;
1140:     T cnt = (T)1 << sz;
1141:     T current_path = 0;
1142:
1143:     for (int i = B - 1; i >= 0; i--) {
1144:         if (basis[i]) {
1145:             cnt >>= 1;
1146:             int bit_x = (x >> i) & 1;
1147:             int bit_path = (current_path >> i) & 1;
1148:
1149:             if (bit_x == 1) {
1150:                 res += cnt;
1151:                 if (bit_path == 0) current_path ^= basis[i];

```

```

1152:             } else {
1153:                 if (bit_path == 1) current_path ^= basis[i];
1154:             }
1155:         } else {
1156:             int bit_x = (x >> i) & 1;
1157:             int bit_path = (current_path >> i) & 1;
1158:
1159:             if (bit_path != bit_x) {
1160:                 if (bit_path < bit_x) return res + cnt;
1161:                 else return res;
1162:             }
1163:         }
1164:     }
1165:     return res;
1166: }
1167: };
1168: // XorBasis<long long> xb;
1169:

```